



h_da

HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

fbeit

FACULTY OF
ELECTRICAL ENGINEERING AND
INFORMATION TECHNOLOGY

Internship Report

An internship report submitted for the degree of

Master of Science (M.Sc.)

to Darmstadt University of Applied Sciences
Department of Electrical Engineering and Information
Technology

submitted by

Suhas Bangalore Mruthunjaya

Matr. Nr. 1119281

Completion date : 31-10-2024

Academic Supervisor : Prof. Dr.-Ing. Thomas Schumann

Company Supervisor : Björn Habrich

Student

Suhas

Bangalore Mruthunjaya

First (Given) Name

Last (Family) Name

22-09-1998

1119281

Date of Birth

Matr.-No.

Prof. Dr.-Ing. Thomas Schumann

Academic Supervisor

In partial fulfillment of the requirements of the **University of Applied Sciences Hochschule Darmstadt (h_da)** for the degree **Master of Science in Electrical Engineering** carried out in collaboration with **Industrial Enterprise**

Company: **qutools GmbH**

Address: **Idstein**

Kreuzgärten 2

65510 Idstein – Germany

- ✓ This Internship report contains confidential data and may only be made available to the supervisor, the members of the examination board and authorized members of Darmstadt University of Applied Sciences.

Declaration

I hereby declare that this report is a presentation of my original work and that no other sources were used other than what is cited.

I furthermore declare that wherever contributions of others are involved, this contribution is indicated, clearly acknowledged and due reference is given to the author and source.

I also certify that all content without reference or citation contained in this report is original work.

I acknowledge that any misappropriation of the previous declarations can be considered a case of academic fraud.

Darmstadt, 31-10-2024

Suhas B M
Suhas Bangalore
Mruthunjaya

Nondisclosure Statement

This report contains proprietary and confidential information that is the property of qutools GmbH. The content is provided exclusively for review by authorized personnel, including supervisors, examination boards, and relevant officials of University of Applied Sciences Hochschule Darmstadt (h_da) in connection with the requirements of the internship and degree program. Unauthorized disclosure, distribution, or reproduction of any part of this report is strictly prohibited without prior written consent from qutools. All data, designs, intellectual property, and insights contained within this report are intended solely for the evaluation and academic assessment process and may not be used for any other purpose.

By reading this document, you agree to maintain the confidentiality of the contents within and acknowledge the proprietary rights of qutools.

Abstract

This report presents the development of a Bell Inequality hardware accelerator and a Python-based Graphical User Interface (GUI) using the qudi framework. The internship was conducted in two parts. The first part involved implementing a hardware accelerator for verifying Bell's Inequality, primarily through FPGA-based VHDL design and simulation. The hardware module utilized finite state machines (FSM) to coordinate complex computation tasks essential for the experiment, ultimately enabling high-speed and reliable data processing. Additionally, the accelerator integrates a custom Division entity, designed to handle the division of experimental data in real-time with precision and efficiency. Through this process, key insights were gained into FPGA architectures, VHDL programming, and embedded system optimizations.

The second part focused on developing a Python-based GUI leveraging the open-source qudi platform, providing users the flexibility to implement custom Python programs and control devices such as quEDU and quNV. This phase introduced complex Python frameworks, modular software design, and concepts related to nitrogen-vacancy centers in diamond, which are used for quantum experiments. The software architecture was built to interface seamlessly between the FPGA core and the GUI layer, providing a user-friendly environment for educational and research-based quantum applications. The report further explores the interdisciplinary integration of quantum mechanics, embedded systems, and software engineering within a real-world industrial setting, highlighting the technical and theoretical learning outcomes.

In this internship, I worked extensively through the five key project phases: requirements engineering, design, implementation, testing, and delivery. While developing the Python GUI, I gained a comprehensive understanding of requirement engineering, design, implementation and testing, balancing usability with functional requirements. In contrast, the Bell Inequality hardware accelerator provided significant experience with design and implementation, particularly through hands-on FPGA programming and system optimization. Mastery of each phase across both projects contributed substantially to personal and professional growth, and the exposure to quantum physics and quantum system applications sparked a strong interest in advanced scientific and engineering concepts that will guide my future endeavors.

Contents

List of Figures	iv
Acronyms	v
I Internship	1
1 Introduction	2
1.1 Motivation	2
2 ABOUT THE COMPANY	3
2.1 VISION	3
2.2 Company Overview	3
2.2.1 Optics	3
2.2.2 Electronics	4
2.2.3 Software	4
2.2.4 Embedded Software	4
2.2.5 Construction	4
2.2.6 Production	4
3 BELL INEQUALITY	5
3.1 Bell's Theorem and the Non-Locality of the universe	5
3.1.1 EPR Paradox	5
3.1.2 John Bell's Experimental Proof Validates Einstein's View of Reality	7
3.2 Quantenkoffer - Quantum Science Suitcase	9
3.3 Realization of Bell's Inequality with the Quantenkoffer	11
3.3.1 Theoretical background	11
3.3.2 Bell's Inequality Hardware Accelerator	13
3.4 Random Number Generators	18
4 Python GUI Using qudi	22
4.1 Objective	22
4.2 quEDU - The Science Kit for Quantum Education	22

4.3	quEDU Key Features	23
4.4	quNV - Nitrogen Vacancy Center Board	24
4.5	Software Architecture of quEDU-NV Python Software	27
4.5.1	Standalone Operation Without qudi	28
4.5.2	qudi's Connector Class for Decoupling and Multithreading	29
4.5.3	Signal-Slot Mechanism Using PySide2	29
4.6	quEDU-NV Python Graphical User Interface Design	30
4.6.1	Home Screen	30
4.6.2	Laser properties	31
4.6.3	Confocal Alignment	32
4.6.4	Optically Detected Magnetic Resonance (ODMR)	34
II	Appendix	36
A	Experimental Realization of Bell's Inequality with the Quantenkoffer	37
A.1	Necessary Components	37
A.2	Experimental description	37
B	quEDU Python GUI Software - Setup	39
B.1	Based on Python Version 3.10	39
B.2	Update installation tools	39
B.3	qudi framework	39
B.4	Continue without qudi framework and virtual python environement	40
B.5	Start quEDU Python Software	40
	Bibliography	41

List of Figures

3.1	Pictorial representation of quantum entangled particles[3]	5
3.2	Pictorial representation of a lab generating two entangled particle with opposite spin [3]	6
3.3	John Bell [3]	7
3.4	Spin Measurement in 3 different Axes of the Entangled pairs[3]	8
3.5	Possible outcomes of the spin measurement[3]	8
3.6	Quantenkoffer[7]	9
3.7	Quantenkoffer(b)[7]	10
3.8	quBricks[7]	11
3.9	The coincidence count values when a Modulo-based counter was used for random event generation (outs_ values)	19
3.10	The coincidence count values when a LFSR was used for random event generation (outs_ / counts_ values)	20
3.11	The coincidence count values when a uniform procedure was used for random event generation (outs_ values)	20
4.1	quEDU [5]	22
4.2	The quEDU manages all experiments and gauges their photon output, while an NV board explores nitrogen vacancy centers in diamond.[4] .	25
4.3	quNV[6]	25
4.4	The intensity of the green laser has been measured using Avalanche Photo Diodes (APDs). The recorded spikes indicate that the laser is operating in sequencer mode, characterized by bursts of laser pulses. Conversely, the steady portion reflects the laser's operation in continuous mode, where it remains consistently activated.	26
4.5	This image illustrates the red fluorescence emitted from the diamond as a result of the interaction between the incoming green laser and the diamond.	26
4.6	The GUI controls the coils to create a 3D magnetic field, while a microwave frequency sweep captures the ODMR spectra of the diamond.[5]	27
4.7	quEDU/NV Python Software Block Diagram	28
4.8	Home Screen	31
4.9	Laser Properties	32
4.10	Confocal Alignment	33

List of Figures

4.11	Target positions are set based on the Diamond identified pixels	34
4.12	ODMR visualization	35
A.1	For the Bell Experiment, put one polarizer in each beam path. You then set the polarizers to different angles and record the coincidence count rate for each setting	38
A.2	Worksheet	38

Acronyms

quEDU Quantum Education Kit

quNV Quantum Nitrogen Vacancy Centers Experiment board

FPGA Field Programmable Gate Array

GUI Graphical User Interface

EPR Einstein-Podolsky-Rosen

CHSH Clauser-Horne-Shimony-Holt

FSM Finite State Machine

ODMR Optically Detected Magnetic Resonance

NV Nitrogen Vacancy

APD Avalanche Photodiodes

Part I

Internship

1 Introduction

1.1 Motivation

The primary motivation for this internship was to deepen technical knowledge in both hardware and software domains through practical, hands-on projects involving quantum technology and embedded systems. Bell's Inequality, a cornerstone in quantum mechanics, provided an ideal platform for exploring FPGA-based acceleration, as it requires both precision in design and speed in processing. This hardware-centric task expanded understanding of VHDL implementation, finite state machines, and FPGA functionality, all of which are critical for high-performance computing tasks within quantum technology. The hands-on experience gained here also enhanced skills in hardware debugging and optimized design strategies, aligning with the industry's demand for efficient quantum experiment systems.

The second phase of the internship focused on developing a user-centric Python GUI based on the qudi framework, motivated by the need for accessible and interactive quantum systems. The GUI aimed to streamline user control over quEDU and quNV devices, providing a customizable platform that allows users to implement their own Python programs and easily interact with hardware. These tools are essential for conducting experiments involving nitrogen-vacancy centers in diamond.

2 ABOUT THE COMPANY

2.1 VISION

"We at qutools are excited about quantum physics and we want to share this fascination with you. The purpose of our activity is to enable the better understanding of quantum physics, on one hand, and advancing technology through this understanding, on the other hand. That is why we focus on innovation, while addressing the needs in your lab. We want you to be able to concentrate on your research, not on the measurement tools. With our knowledge of the technology, the hardware and the software needed to perform your type of experiment, together we will find the best solution to facilitate your research and we will tailor it to your needs"

2.2 Company Overview

qutools is a leading technology company specializing in advanced optics, electronics, software, and construction solutions. Founded in 2005, qutools has established a reputation for delivering high-quality products tailored to the specific needs of its clients. The company's commitment to excellence is demonstrated through its long-term development expertise, scientific understanding, and honest consultation at the highest level.

2.2.1 Optics

qutools has extensive experience in building demonstrators for entangled photons, fiber optics, and single photon sources. Their expertise extends to interferometric length measurements, color centers in diamond, quantum dots, and fluorescence spectroscopy.

2.2.2 Electronics

The company's skills in analog and mixed-signal electronics enable ultra-fast time measurements on the picosecond scale, with customizable interfaces and ports available on demand.

2.2.3 Software

qutools develops software for data collection and analysis, offering intuitive GUIs across all operating systems. Their capabilities include controlling complex systems like atomic force microscopes through tablet apps.

2.2.4 Embedded Software

With over 15 years of experience in logic programming, qutools excels in algorithmic control, ultra-fast time detection, microscopy, and interferometry.

2.2.5 Construction

Their design expertise ranges from simple mirror mounts to complex mechanical components, ensuring precision and reliability in every product.

2.2.6 Production

In-house production and assembly of high-quality parts, including the machining of aluminum, steel, non-ferrous metals, and plastics, enable qutools to bring any design to life. The final assembly processes integrate optical, electrical, and mechanical components to deliver seamless solutions

qutools takes pride in its ability to customize products to meet the specific requirements of each application, ensuring optimal performance and customer satisfaction.

3 BELL INEQUALITY

3.1 Bell's Theorem and the Non-Locality of the universe

This dates back to 1935, when Albert Einstein and his colleagues Boris Podolsky and Nathan Rosen together published a paper named “**Can Quantum-Mechanical Description of Physical Reality be Considered Complete?**”[1]

3.1.1 EPR Paradox

The fundamental conclusion reached in the paper[1] authored by Einstein and his colleagues was not that quantum mechanics are erroneous in terms of making predictions that contradict observations or experiments. Instead, they asserted that the reality depicted by quantum mechanics is an incomplete portrayal of reality. They contended that a more profound explanation must exist, even though they were unable to provide such an explanation. They believed that the paper established the necessity of uncovering this deeper understanding.

Their analysis was based on the concept of quantum entanglement, which is illustrated by the idea that two particles can be widely separated and yet exhibit a connection as if there is an invisible quantum mechanical thread that links them together.

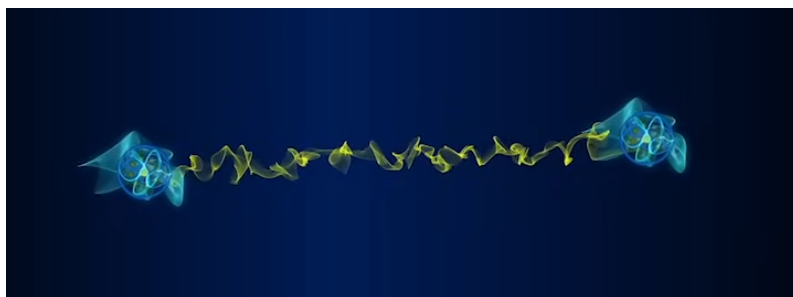


Figure 3.1: Pictorial representation of quantum entangled particles[3]

3 BELL INEQUALITY

In standard quantum mechanics, entangled particles exhibit a phenomenon where a change in one particle, for example the one on the right, can immediately impact the measurement of the particle on the left, despite being separated by significant distances. This behavior, characterized by Einstein as “**Spooky action at a distance**”, presents a peculiar and intriguing aspect of quantum mechanics.

Let’s delve further into the EPR paradox and explore the points of contention that Einstein and others had regarding the quantum mechanical spin of particles. We can imagine that we have a couple of detectors and one detector on the right, one detected on the left and in the middle there is a device that will send out particles that are entangled.



Figure 3.2: Pictorial representation of a lab generating two entangled particle with opposite spin [3]

Now when these entangled pairs reach the detectors, the actual measurement of their spin is calculated. The spookiness is that according to quantum mechanics, the particles are always in a fuzzy mixture of spin orientation and yet upon measurement they both disambiguate that both choose a definite reality even though they’re far apart, they’re somehow able to correlate the outcomes. If one particle is found to be spin up upon measurement, its entangled counterpart inevitably displays spin down, and vice versa.

Einstein contended that particles do not exist in a state of ambiguous up-and-down spin until they reach the detectors, only when measured do they definitively assume a particular state. He argued that this implied some sort of mysterious interaction between these widely separated particles, which he believed was not how the world functioned. According to Einstein, if a detector detects a particle as spinning up, then it always was spinning up; it never existed in the indeterminate state depicted by the mathematics of quantum mechanics. Consequently, the trio asserted in their paper[1] that the mathematics of quantum mechanics is incapable of providing a

complete and comprehensive description of reality.

3.1.2 John Bell's Experimental Proof Validates Einstein's View of Reality

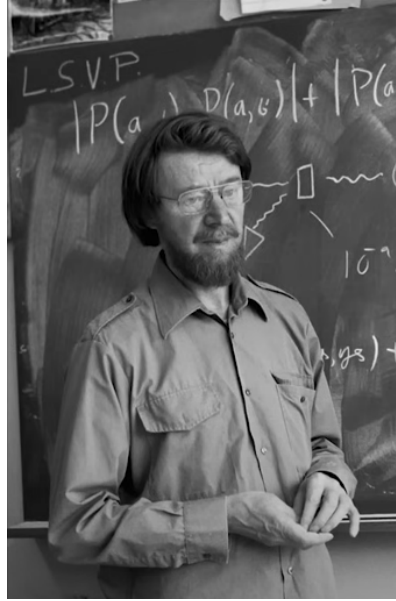


Figure 3.3: John Bell [3]

In 1964, John Stuart Bell, a physicist from Northern Ireland, authored a paper[2] entitled “**ON THE EINSTEIN PODOLSKY ROSEN PARADOX**”. He writes in his paper[2] that shows the Einstein’s view of reality, in which particles are always definite that yields a measurable difference is testable.

Let us consider the previous experiment with two detectors and a device that send out entangled particles. But this time we are using an upgraded detectors.i.e., they don’t just measure the spin along one axis, but these can measure the particle spin about three different axes that are 120° from each other. According to Einstein’s perspective on reality, particles inherently possess a fixed value, more like a concealed value even before measurement. This concept is widely known as local realism. Embracing the local realism phenomenon, particles are required to have specific spins along all three possible axes.

There is now the possibility of measuring the same spin in the detectors when measured at two different axis. For instance, if the first entangled particle is measured on axis 1, a spin up is obtained at the left detector and likewise, a spin up is obtained at the right detector when measured on axis 2. Bell argues that if this experiment is

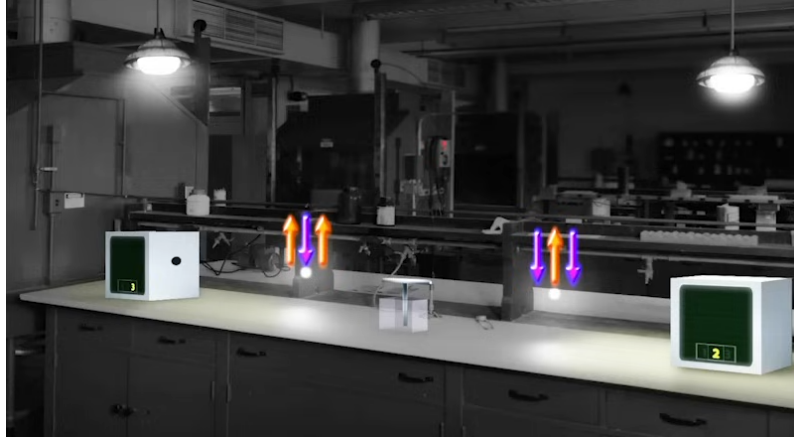


Figure 3.4: Spin Measurement in 3 different Axes of the Entangled pairs[3]

repeatedly carried out with the detector settings being randomly selected for each trial, the instances where, the detectors register opposite spins will occur at least $5/9$ of the time in the overall outcomes. In the given scenario, there are clearly 9 potential results, and the spins are opposite in 5 out of 9 occurrences.

In the context of two particles and three axes, it is reasonable to deduce that at least two of the spin directions must be identical. There are scenarios where all three axes share the same spin orientation (such as up, up, up on the left particle and down, down, down on the right particle). In this particular scenario, all 9 outcomes yield opposite spin results for 9 possible measures. Thus, the probability of the result indicating opposite spins is five out of nine when all the spins along the three axes are not the same and nine out of nine when they are the same.



Figure 3.5: Possible outcomes of the spin measurement[3]

In repeated experiments involving random entangled particles and random detector

settings conducted over a large number of iterations, it is expected that at least 5 out of 9 times, or approximately 55% of the time, opposite spin results will be obtained. This aligns with Einstein’s belief that particles possess definite spins before measurement. However, empirical evidence suggests that the frequency of obtaining opposite spins is considerably lower, around 50%, contradicting Einstein’s perspective. Instead, this outcome supports the standard quantum mechanics calculation, which indicates that particles exist in a nebulous combination of spin states rather than in a predetermined state.

The whole idea behind EPR Paradox and Einstein’s view of hidden variables theory was to rule out the “**Spooky action**” in between and non-locality. However, the results of experiments suggest otherwise.

3.2 Quantenkoffer - Quantum Science Suitcase

The Quantenkoffer, an innovative device, designed and developed by qutools GmbH, serves as a portable and user-friendly quantum photonics laboratory for conducting a wide range of experiments spanning a century of quantum physics. Its primary strength lies in its adaptability to both generate and detect visible laser light, single photons, and even entangled pairs. This is made possible through the integration of optical elements, mechanical components, and digital circuits, enabling a comprehensive coverage of experiments and topics within the realm of quantum physics.



Figure 3.6: Quantenkoffer[7]

The Key features of the Quantenkoffer [7]:

- **Experiment Flexibility:** On the high-precision board with 86 slots various setups can be individually created by placing the optical tokens into the electromechanical slots. Conducting well-known photonics experiments or setting up new ones can be as easy as playing.
- **Motorized Adjustment:** The possible automatic adjustment through actuators and sensors facilitates the setup of experiments and the quick realization of measurements, allowing for the playful immersion into the fascinating world of quanta.
- **Single Photons, Entangled Pairs:** A set of lasers and optical crystals form the core of the photon source, which the user can view by flipping the optical setup board. Single pairs of photons are generated through nonlinear processes within optical crystals. Each element of the precise source can be adjusted by the user or automatically optimized by the Quantenkoffer. Additional red laser light follows the path of the single photons for proper adjustment of the optical setups.



Figure 3.7: Quantenkoffer(b)[7]

- **Intuitive Operation** After the automatic detection of the inserted tokens, a setup display appears on the touch-pad. The user-friendly software supports the user in choosing from a wide range of display-widgets, four of which are

active on the large monitor.

- **Picosecond Range Timing:** The ultra-fast processing of the measured events, enabled through powerful digital technology, allows the detection of time differences in the picosecond range, for example to measure the speed of light on the board with few tokens.
- **Optical Tokens - the quBricks:** A variety of optical tokens can be placed on the high-precision board and used in different experiments. The Quantenkoffer identifies, reads, and digitally manages the markers through the electronics surrounding the partially motorized optical elements. Additionally, a touch sensor on top of each quBrick adds physical engagement to enhance cognitive activation during the learning process.



Figure 3.8: quBricks[7]

3.3 Realization of Bell's Inequality with the Quantenkoffer

3.3.1 Theoretical background

The most widely used version of Bell's inequality in experimental tests is the one formulated by Clauser, Horne, Shimony, and Holt (referred to as CHSH). The key concept behind the CHSH-Bell inequality is that in local realistic theories, the absolute value of a specific combination of correlations between two particles is constrained by 2. The inequality can be expressed as:

$$S(\alpha, \alpha', \beta, \beta') = [E(\alpha, \beta) + E(\alpha', \beta) - E(\alpha, \beta') + E(\alpha', \beta')] \leq 2$$

3 BELL INEQUALITY

where $\alpha(\alpha')$ and $\beta(\beta')$ denote the local measurement settings of two observers, each receiving one of the particles.

The coincidence count rate for the combination of polarizer settings $C(\alpha, \beta)$ is defined, and the normalized expectation value $E(\alpha, \beta)$ of the correlations between the measurements is given by:

$$\frac{C(\alpha, \beta) - C(\alpha, \beta_p) - C(\alpha_p, \beta) + C(\alpha_p, \beta_p)}{C(\alpha, \beta) + C(\alpha, \beta_p) + C(\alpha_p, \beta) + C(\alpha_p, \beta_p)}$$

where α_p and β_p are the polarization perpendicular - that is rotated by 90° - with respect to α and β .

Measurements of the coincidence count rate with entangled photons reveal a sinusoidal dependence, described by $C(\alpha, \beta) \propto \cos 2(\theta)$, where θ is the angle difference between the polarizer orientations. This behavior predicts that the correlation coefficients follow the form $E(\alpha, \beta) = \cos(2\theta)$. The CHSH inequality is violated for various combinations of orientations. The maximum violation occurs under specific conditions, particularly when the settings are

$$(\beta - \alpha) = (\alpha' - \beta') = (\beta' - \alpha') = 22.5^\circ \text{ and } (\beta' - \alpha) = 67.5^\circ,$$

giving the upper limit on the violation of the CHSH inequality, $S = 2\sqrt{2}$. Quantum entanglement thus results in stronger correlations that surpass the local realistic limit of 2, necessitating a reconsideration of at least one of the foundational concepts—locality or realism. Locality posits that local events cannot be influenced by actions in space-like separated regions, while realism suggests that an external reality exists independently of observation.

In the experiment, any set of orientations satisfying the above conditions can be selected. For instance, the angles can be set to $\alpha = 0^\circ, \alpha' = 45^\circ, \beta = 22.5^\circ$ and $\beta' = 67.5^\circ$. Four separate experimental runs are conducted corresponding to the four terms $E(\alpha, \beta)$ in the direction of S. Each term $E(\alpha, \beta)$ is calculated from the four coincidence measurements, resulting in a total of 16 count rates to be called experimentally.

Due to the statistical nature of the inequality, it is essential to choose a sufficiently long integration time to accumulate the necessary 16 coincidence rates. The standard deviation of the experimental value S can be computed using:

$$\Delta S(\alpha, \alpha', \beta, \beta') = \sqrt{\sum_{a=\alpha, \alpha'} \sum_{b=\beta, \beta'} \Delta E(a, b)^2},$$

where the errors $\Delta E(a, b)$ on the individual correlation coefficients are computed via Gaussian error propagation. If the experimental value S is numerically greater than

2 it has violated the CHSH inequality. The strength of violation is usually defined in terms of the number n of standard deviations which add up into the gap between the experimental value S and the local realistic bound 2:

$$n_{\Delta} = \frac{S - 2}{\Delta S}$$

The experimental demonstration of Bell's inequality using the Quantenkoffer can be found in the Appendix: A

3.3.2 Bell's Inequality Hardware Accelerator

The Bell inequality hardware accelerator is designed to compute and verify experimental results related to the Bell inequality theorem, which plays a fundamental role in quantum mechanics by testing whether quantum phenomena can be explained by classical physics. The hardware accelerator coordinates multiple actors in the system, processes quantum events, and performs high-speed computations for the experiment using an embedded state machine (FSM) to handle the control flow efficiently. In this explanation, I will describe the system's architecture, state machine (FSM), division process, and key signals.

System Architecture:

The main entity `experiment_bell` receives multiple inputs and controls, including clock signals (`clk`), millisecond timers (`msec`), parameters for the experiment (`param`), and event signals. The core output is the result of the experiment (`result`), which is stored in a custom type `t_param_inst`. It interacts with two actors in the system, identified by their positions (`actor_positions`), statuses (`actor_statuses`), and their dynamically assigned IDs.

Key internal signals include angles for both actors (`param_angle_a1`, `param_angle_a2`, `param_angle_b1`, `param_angle_b2`) and various signals that control the division and calculation process, which are essential for computing results related to the Bell inequality.

The division process is handled by a custom division entity `Divide_entity`, which takes in a dividend and divisor, and produces a quotient, used to accumulate results based on quantum measurements.

Finite State Machine (FSM):

The core control logic of the system is managed by a finite state machine (FSM) with several key states that dictate the experiment's flow. The FSM coordinates the initialization of the actors, the application of angle settings, event handling, and the final measurement and evaluation phases.

The main FSM states include:

- **s_actor_init1 and s_actor_init2:**
In these initialization states, the system assigns actor IDs (`actor_ids`) and requests actor ID changes via `actor_id_change_reqs`. This step is crucial for assigning the correct experimental roles to each actor. The state machine waits for acknowledgment (`actor_id_change_grants`) from the actors before moving to the idle state.
- **s_idle:**
In this state, the system is idle, awaiting the start command (`cmd_bell_start`). When the experiment is initiated, the FSM moves to the next phase, resetting relevant counters and control signals. The idle state is also used as an abort mechanism, where the experiment can reset if an abort command (`cmd_abort`) is issued.
- **s_set_angles:**
This state assigns the angles for both actors (polarizers) in preparation for the experiment. The angles for actor A and actor B are set based on the current step in the experiment. The system updates `actor_targets` with the required angle values, and the state machine waits until both actors reach their targets (`x_actor_target_reached`, `y_actor_target_reached`).
- **s_wait_for_actors:**
Once the target angles are set, the system waits for the actors to reach their positions. This state ensures synchronization between the physical actors and the control logic before proceeding.
- **s_wait_msec:**
A timing state, where the system waits for the millisecond signal (`msec`) to synchronize with the experimental clock, ensuring proper exposure time for measurements.
- **s_measure:**
During the measurement phase, the FSM increments counts based on the selected event (`event_sel`). These counts represent the detection of quantum events and are crucial for later calculation of the Bell inequality violation.
- **s_eval_1:**
After measurement, the system moves into the evaluation state, where it performs division to calculate the Bell inequality result. The division operation, handled by a custom division module, computes the ratio of counts collected in `s_measure`. This step is critical for evaluating whether the Bell inequality has been violated or satisfied. The FSM accumulates the results, taking into account the sign of the dividend to handle negative results properly.

- **s_error:**

The error state is activated if any issues occur during the experiment, such as actors not reaching their targets or other operational errors. This state halts the experiment and resets the control logic.

The state transitions are largely driven by key signals such as `cmd_bell_start`, `msec`, and the actor statuses. The FSM ensures that all required actions, from initialization to final evaluation, are completed in a structured manner without missing critical steps.

Division and Calculation Process:

The module, named `Divide_entity`, is designed to perform binary division through an iterative process using a state machine for control. The goal of this implementation is to enable division between two unsigned fixed-point numbers, yielding a quotient after multiple clock cycles.

→ **Entity Definition:**

The module's interface is defined in the entity `Divide_entity`, where the key inputs include a clock signal (`clk`), a start signal (`start`), the divisor, and the dividend, both of which are unsigned numbers of size `WIDTH` bits. The output signals include the quotient, which is $2 + (2 * WIDTH)$ bits wide, and a done signal that indicates when the division process is completed.

The `WIDTH` parameter, which is a generic integer, allows the bit-width of the divisor and dividend to be configured. This parameterization makes the design flexible and scalable for different bit-widths.

→ **Architecture Overview:**

The internal implementation resides in the behavioral architecture. The division is performed using a binary long-division algorithm, where each division step shifts and subtracts in a manner similar to the manual long division of integers, but here, the numbers are represented in binary format.

The architecture uses a state machine to control the division process, ensuring synchronization with the clock and proper sequencing of operations.

→ **Key Signals:**

Several internal signals are declared within the architecture:

- `temp_divisor`: Holds the divisor used during the division process.
- `temp_dividend`: Stores the dividend, extended to twice the width of the original dividend to accommodate the shifting process.

- temp_quotient: Stores the result of the division as bits are calculated in each iteration.
- zero_pad: Used to initialize the extended dividend by padding zeros at the beginning.
- count: Tracks the number of iterations during the division process.

→ State Machine Design:

The division process is controlled by a finite state machine (FSM) with three states:

- Idle State(s_idle): In this state, the module waits for the start signal to initiate the division. When the start signal is asserted, the divisor and dividend are loaded into temp_divisor and temp_dividend respectively, and the quotient register (temp_quotient) is cleared. The state machine then transitions to the s_calculate state.

```

1  when s_idle =>
2      done <= '0';
3      if start = '1' then
4          temp_dividend <= zero_pad & dividend;
5          temp_divisor <= divisor;
6          temp_quotient <= (others => '0');
7          state <= s_calculate;
8      end if;

```

- Calculate State (s_calculate): This is the core of the division algorithm. At each clock cycle, the higher bits of the temp_dividend (representing the current remainder) are compared to the divisor (temp_divisor). If the current remainder is greater than or equal to the divisor, a subtraction is performed, and a '1' is shifted into the quotient. Otherwise, the remainder is shifted left without subtraction, and a '0' is shifted into the quotient. The count signal keeps track of how many iterations have been performed, and the process continues until the full quotient is determined.

```

1  when s_calculate =>
2      count <= count + 1;
3      if temp_dividend((2 * WIDTH)-1 downto WIDTH) >=
4          temp_divisor then
5          sub_sig := temp_dividend((2 * WIDTH)-1 downto
6              WIDTH) - temp_divisor;
7          temp_dividend <= shift_left(sub_sig &
8              temp_dividend(WIDTH-1 downto 0), 1);
9          temp_quotient(((2 + (2 * WIDTH))-1) - count)
10             <= '1';
11      else

```

```

8      temp_dividend <= shift_left(temp_dividend,1);
9      temp_quotient(((2 + (2 * WIDTH))-1) - count)
      <= '0';
10     end if;
11     if count = ((2 + (2 * WIDTH))-1) then
12         state <= s_done;
13     end if;

```

- **Done State (s_done):** Once all the necessary bits of the quotient are computed, the FSM enters the s_done state. Here, the final quotient is output to the quotient signal, and the done signal is asserted to indicate the completion of the operation. Afterward, the state machine resets the counter (count) and returns to the idle state (s_idle), awaiting a new division request.

```

1     when s_done =>
2         quotient <= temp_quotient;
3         done <= '1';
4         count <= 0;
5         state <= s_idle;

```

The state machine ensures that the division operation is synchronized with the clock and that all necessary steps are performed in a sequential manner. The FSM is triggered on the rising edge of the clock, and the division process takes several clock cycles, depending on the bit-width of the inputs.

→ Binary Division Algorithm

The binary division algorithm works by shifting and subtracting the dividend by the divisor. Each iteration of the algorithm examines the highest bits of the current remainder (a portion of the extended dividend). If the remainder is greater than or equal to the divisor, the divisor is subtracted from the remainder, and a '1' is placed in the quotient. Otherwise, a '0' is placed in the quotient, and the remainder is simply shifted left. This process is repeated until all bits of the quotient are computed.

In this design, the quotient is generated over $2 + (2 * WIDTH)$ clock cycles, where $WIDTH$ is the bit-length of the inputs. This number of cycles allows the complete quotient to be generated with precision, accounting for any fractional parts resulting from the division.

The division module described here utilizes a state machine to perform binary long division over multiple clock cycles. By employing fixed-point arithmetic, the module can divide two unsigned numbers of configurable bit-width, yielding a quotient that includes both integer and fractional parts.

3.4 Random Number Generators

To simulate the event ticks, which typically increase with a rise in coincidence counts at the Avalanche Photodiodes, it was necessary to randomly enable the event_sel signal within the s_measure state. Four separate experimental runs are conducted corresponding to the four terms $E(\alpha, \beta)$ in the direction of S. Each term $E(\alpha, \beta)$ is calculated from four coincidence measurements, resulting in a total of 16 count rates. These coincidence counts are subsequently used in calculating the S-value. In the testbench, random number generators were employed to imitate the random generation of events. The types of RNGs that were evaluated include:

- **Fixed High-Low Intervals with wait for Statements:** In this method, fixed wait intervals are used to alternate tb_events between high and low states.

```
1      tb_events <= (others => '1');
2      wait for 2000ns;
3      tb_events <= (others => '0');
4      wait for 4000ns;
```

- **Mechanism:** This approach explicitly sets tb_events to high for 2000 ns, followed by low for 4000 ns.
- **Purpose:** Using fixed timing intervals creates a deterministic high-low pulse sequence, simulating a periodic signal but with zero randomness.
- **Result:** This method lacks randomness due to the constancy of the interval, resulting in identical rates for the four coincidence counts. The issue with having the same count number is that the normalized expectation value will be zero, thereby preventing any violation of Bell's Inequality. [Refer: 3.3.1]

- **Modulo-based Counter Check:** The second approach involves using a counter that increments on every clock cycle and triggers an event based on a modulo operation.

```
1      if rising_edge(tb_clk) then
2          event_cnt := event_cnt + 1;
3          if event_cnt mod 7 = 0 then
4              tb_events <= (others => '1');
5          else
6              tb_events <= (others => '0');
7          end if;
8      end if;
```

- **Mechanism:** Here, the event_cnt variable increments with each clock cycle. When the count is a multiple of 7 (i.e., $\text{event_cnt} \bmod 7 = 0$),

tb_events is set to high ('1'), otherwise it's set low ('0').

- **Purpose:** This approach introduces a regular pattern of triggering tb_events, creating a somewhat random-looking pulse.
- **Result:** However, because this operation relies on the modulo of a fixed integer, it produces a fixed frequency that is predictable and periodic, hence, lacking the desired randomness, which results in the same count number for the coincidence counts.

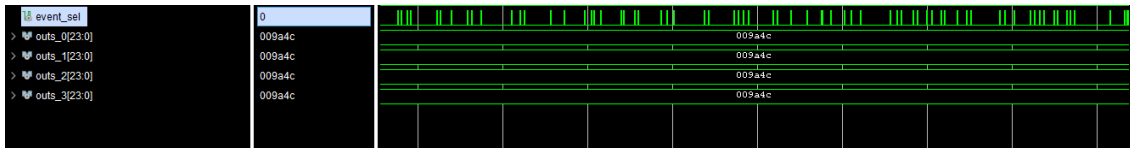


Figure 3.9: The coincidence count values when a Modulo-based counter was used for random event generation (outs_ values)

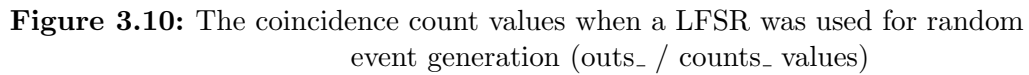
- **Linear Feedback Shift Register(LFSR) Implementation** LFSR generates a pseudo-random sequence. It is a shift register where each bit is a linear function of its previous bits, typically using XOR operations for feedback.

```

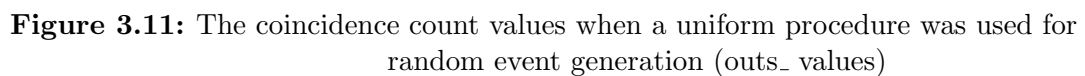
1      if rising_edge(tb_clk) then
2          if reset = '0' then
3              Q_lfsr <= x"01";
4          else
5              tmp := Q_lfsr(0) XOR Q_lfsr(2) XOR Q_lfsr(3) XOR
6                  Q_lfsr(4);
7              Q_lfsr <= tmp & Q_lfsr(7 downto 1);
8          end if;
9      end if;

```

- **Mechanism:** This configuration initializes Q_lfsr to a seed value on reset. On each clock cycle, it performs XOR operations on select bits (Q_lfsr(0), Q_lfsr(2), Q_lfsr(3), and Q_lfsr(4)) to generate the new LFSR value.
- **Purpose:** LFSRs are commonly used for random signal generation due to their simple and fast implementation.
- **Result:** It often repeats with a fixed periodicity, which can make the randomness feel limited for event generation.



- **Mechanism:** In this process, uniform generates a real number r at each clock cycle. When r is above a threshold (e.g., 0.9), `tb_events` is set high, simulating an event.
- **Purpose:** This threshold-based approach introduces randomness effectively, as r changes dynamically each cycle.
- **Result:** By setting the threshold, we control the frequency of event occurrences while keeping a natural randomness in when these events happen. This approach is generally more effective for simulating random events in digital systems. The coincidence counts showed better randomness between each other. This was needed to simulate the violation of Bell Inequality.



```
1      event_occurance : process(tb_clk)
2      variable r : real;
3      variable seed1 : positive := 999;
4      variable seed2 : positive := 999;
5      begin
6      if rising_edge(tb_clk) then
7      uniform(seed1, seed2, r);
8      if r > 0.9 then
9      tb_events <= (others => '1');
10     else
11     tb_events <= (others => '0');
12     end if;
13     end if;
14     end process;
```

4 Python GUI Using qudi

4.1 Objective

The objective of the second phase of the internship was to develop an embedded system interface enabling users to control the quEDU system according to their specific requirements. This involved the creation of a Python API using the clientLib, which provides a range of control functions for device management. Furthermore, the project was extended by developing a Python-based graphical user interface (GUI) to offer a more intuitive and user-friendly experience. The integration and functionality of both the API and GUI were tested using custom examples to ensure seamless operation and user satisfaction.

4.2 quEDU - The Science Kit for Quantum Education

The quEDU serves as a comprehensive platform for conducting a wide range of quantum physics experiments. It is equipped with single photon detectors, time tagging electronics for data acquisition and analysis, as well as digital interfaces designed to manage the experiments effectively.

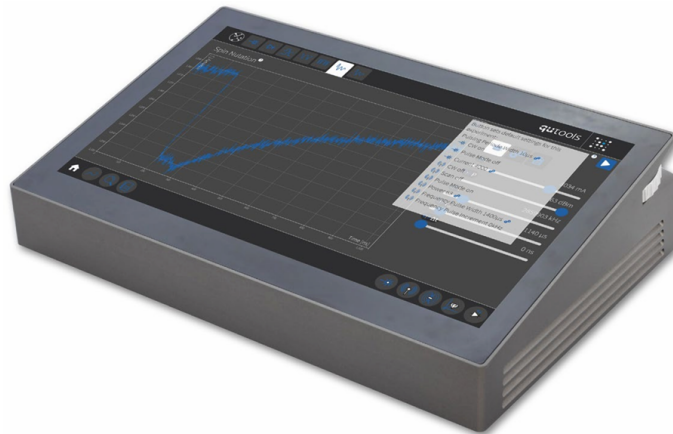


Figure 4.1: quEDU [5]

Moreover, the quEDU, along with its experiment boards, integrates the latest advancements in quantum optics technology to provide an accessible system for academic, research, and applied purposes with remarkable precision. Furthermore, advanced models tailored for scientific applications are also available, delivering exceptional performance to meet the demands of cutting-edge physics experiments.

The qutools Quantum Education kit is specifically tailored for educators, offering a user-friendly and dependable solution to elucidate the intricate concepts of quantum mechanics. This is achieved through the generation and analysis of polarization-entangled photon pairs, as well as the manipulation of electron spins in a diamond.

4.3 quEDU Key Features

The quEDU system from qutools GmbH offers a highly modular and user-friendly platform designed for hands-on exploration of quantum phenomena. Its versatility allows users to conduct a variety of quantum experiments with ease, making it an ideal tool for both educational and research purposes. Below is a summary of its key features and specific applications:

- **Hands-on Study of Quantum Entanglement:** The quEDU system enables practical experimentation with quantum entanglement, providing an intuitive platform for users to explore the core concepts of quantum mechanics.
- **User-Friendly Operation and Complete System** Designed for ease of use, the system features a touchscreen interface and rotary buttons for intuitive control. It is equipped with all necessary components for a complete experimental setup, minimizing the need for additional hardware.
- **Expandability with Add-ons** To further extend its capabilities, quEDU offers add-ons that allow users to expand their experimental scope, making it a versatile tool that grows with the user's needs.
- **Specific Applications**
 - **Student Lab Course Experiments:** Ideal for educational settings, quEDU provides students with hands-on experience in quantum mechanics.
 - **Violating Bell's Inequalities:** The system supports experiments that demonstrate the violation of Bell's inequalities, a key concept in proving the non-classical nature of quantum mechanics.
 - **NV Center Spin Manipulation:** The system allows for experiments involving NV centers (Nitrogen Vacancy centers in diamond) for spin

manipulation, often used in quantum sensing and quantum memory applications.

- **Optically Detected Magnetic Resonance(ODMR):** It also supports ODMR experiments, which are essential for studying the properties of quantum systems under different magnetic fields.

- **Control and Usage**

- **TouchScreen and Rotary Buttons:** Easy control through an intuitive interface.
- **6 Ports for Experiment Boards:** Flexible connections to accommodate various experimental modules.
- **Remote Desktop via WLAN, Ethernet:** The system can be accessed remotely, allowing for convenient control and monitoring through WLAN or Ethernet connections.

- **Detection and Analysis**

- **4 Temperature - Controlled APDs:** The system includes Avalanche Photodiodes (APDs) with fiber ports for precise photon detection, featuring temperature control to ensure stability during measurements.
- **Time tagging Electronics:** Equipped with time tagging electronics with picosecond precision, enabling accurate measurement of time-correlated photon events.
- **Singles and Coincidence Counting:** Provides the ability to count both single and coincidence photon events, crucial for many quantum experiments.
- **Analysis and Data Processing:** Includes tools for analyzing and processing measurement data, streamlining the workflow from experiment to result interpretation.

4.4 quNV - Nitrogen Vacancy Center Board

The NV experiment board provides an opportunity to explore the characteristics of nitrogen vacancy centers in diamond. This includes the examination of NV center fluorescence, the manipulation of electron spins, the application of optically detected magnetic resonance, and the sensing of magnetic fields in samples.

- **Setup with quEDU:** The board must initially be connected to the quEDU

using a DisplayPort cable, which serves both as a power supply and for data communication. Two optical fibers link the NV center board to the avalanche photodiodes (APDs) of the quEDU. One of these fibers is responsible for collecting the red fluorescence emitted by the NV center through the objective, subsequently transmitting it to one of the APDs. The second fiber is utilized to convey a portion of the green excitation light, facilitating closed-loop control of the laser.

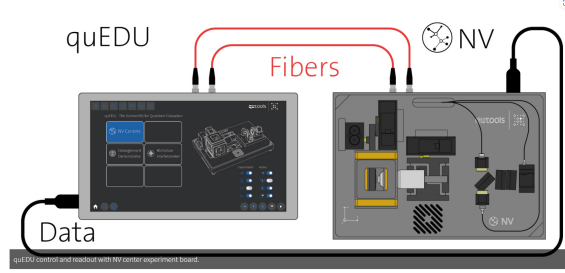


Figure 4.2: The quEDU manages all experiments and gauges their photon output, while an NV board explores nitrogen vacancy centers in diamond.[4]



Figure 4.3: quNV[6]

→ **Laser and Sample Alignment:** The subsequent task involves the precise alignment of the diamond in front of the microscope objective. We commence by activating the powerful green laser located beneath the orange safety cover. Utilizing the alignment controls available in the graphical user interface, we can adjust the sample's position to ensure it is perpendicular to the direction of the laser beam. Ultimately, we observe the red fluorescence emitted by the nitrogen-vacancy (NV) center through the safety glass. Fine alignment in all three dimensions is achieved by plotting the data obtained from the avalanche photodiode (APD) on the quEDU screen.

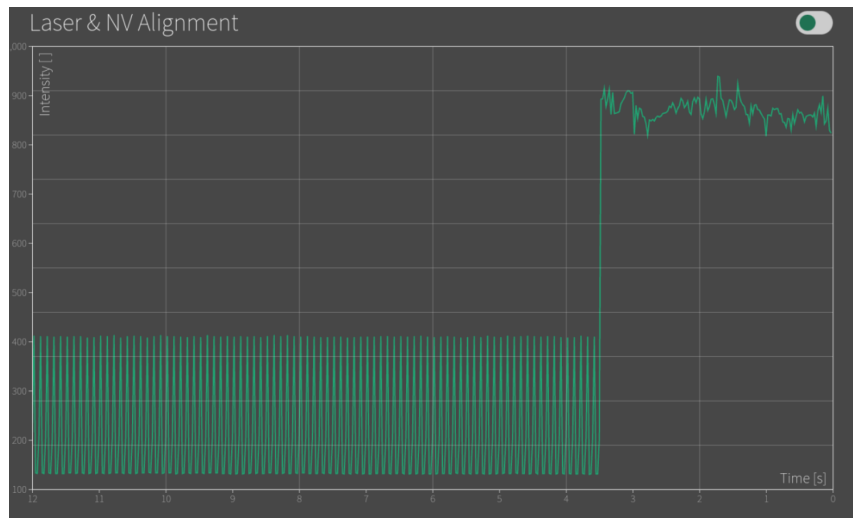


Figure 4.4: The intensity of the green laser has been measured using Avalanche Photo Diodes (APDs). The recorded spikes indicate that the laser is operating in sequencer mode, characterized by bursts of laser pulses. Conversely, the steady portion reflects the laser’s operation in continuous mode, where it remains consistently activated.



Figure 4.5: This image illustrates the red fluorescence emitted from the diamond as a result of the interaction between the incoming green laser and the diamond.

→ **ODMR Scan:** To initiate an ODMR (optically detected magnetic resonance) scan focused on studying the resonance frequencies, it is essential to achieve a high fluorescence intensity. The process involves activating the microwave source, systematically scanning its frequency, and subsequently plotting the resulting fluorescence data. This procedure typically reveals at least two distinctive dips in the observed fluorescence spectrum. Furthermore, by applying a magnetic field through the coils, we can induce a further splitting of the resonance peaks, a phenomenon known as Zeeman splitting.

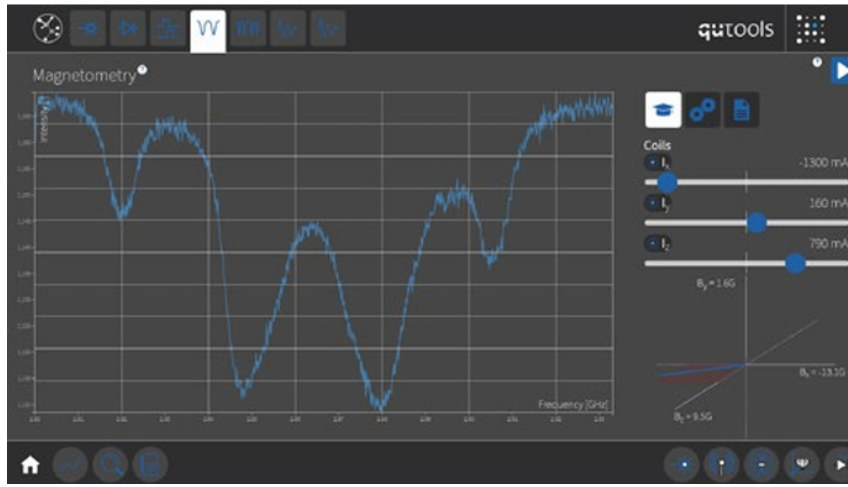


Figure 4.6: The GUI controls the coils to create a 3D magnetic field, while a microwave frequency sweep captures the ODMR spectra of the diamond.[5]

4.5 Software Architecture of quEDU-NV Python Software

The quEDU Python software is developed to manage quantum experiments by leveraging the qudi framework. qudi's architecture promotes modular design, making it well-suited for complex scientific applications by enforcing clear separation between components. This enables better maintainability and easier scaling for a variety of quantum experiment setups. The core of the quEDU system adopts qudi's structural model, which centers on three primary components: the GUI, Logic, and Hardware modules.

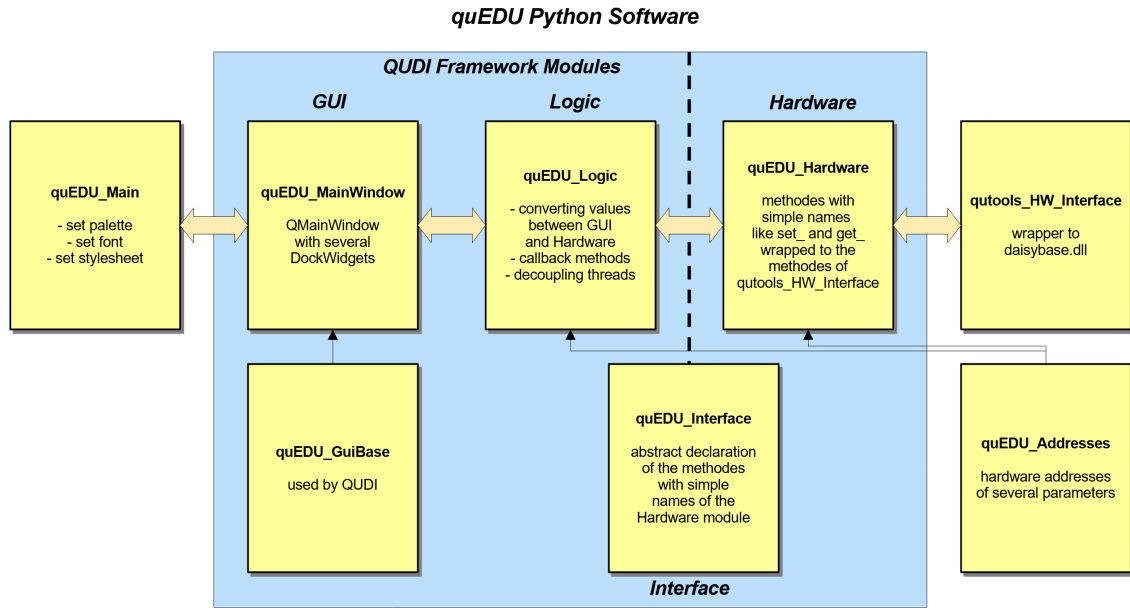


Figure 4.7: quEDU/NV Python Software Block Diagram

- **GUI (Graphical User Interface) Module:** This component handles the graphical elements and user interactions, enabling intuitive access to experimental controls and data displays.
- **Logic Module:** Handles the core functionality and the business logic, coordinating data processing and management.
- **Hardware Module:** Interfaces directly with quantum experiment hardware via the qutools_HW_Interface.py wrapper, which accesses the qutools daisybase.dll.

The GUI module provides interactive widgets for user input, while the Logic module performs calculations or manipulations based on user inputs. The Hardware module acts as the bridge between the software and the experimental hardware, enabling direct communication with hardware elements.

4.5.1 Standalone Operation Without qudi

When running independently of qudi, the quEDU software utilizes a main module, quEDU_Main.py, which serves as the entry point to start the application. This main module initializes each component separately and launches the software.

In the absence of qudi, all modules connect directly, potentially resulting in a tightly coupled structure. This could cause issues in a standalone environment, as all mod-

ules would be operating in a single thread, with both Logic and Hardware operations potentially blocking the GUI and impacting responsiveness. To mitigate this, qudi introduces a unique architectural feature for linking modules and managing communication effectively.

4.5.2 qudi's Connector Class for Decoupling and Multithreading

One of the key architectural elements of qudi is the connector class, which decouples modules by avoiding direct imports. Instead, module connections are configured through the quEDU.cfg file, allowing qudi to dynamically link them at runtime.

qudi also separates modules into different threads, preventing the Logic and Hardware components from blocking the GUI thread. This multithreading ensures a responsive interface, even when complex hardware or logic operations are active.

4.5.3 Signal-Slot Mechanism Using PySide2

qudi leverages Python Qt modules, specifically PySide2, to facilitate communication between modules. PySide2, a Python binding of the Qt framework, provides a powerful signal-slot mechanism for inter-module communication that fits naturally into qudi's architecture.

In the signal-slot pattern, different components communicate through signals (events) and slots (reactions), allowing an event in one module to trigger a response in another. This design enables efficient, asynchronous communication between the GUI and Logic modules, contributing to both decoupling and responsiveness.

- **Signal Transmission:** The GUI module consists of various interactive widgets, such as buttons, sliders, and text fields. Each widget can emit signals in response to user actions, such as `editing.finished`, `button.clicked`, or `slider.moved`. When a user interacts with a widget, it sends out a signal containing the current value or state. This signal is directed to the Logic module, where it might trigger further processing or a value conversion, depending on the type of action performed.
- **Logic to GUI Communication:** After processing the input, the Logic module may send updated values or responses back to the GUI. This is done through additional signals emitted by the Logic module. For instance, if a certain computation is triggered in the Logic module due to a user action, the result is then packaged in a signal and sent back to the GUI, where the corresponding slots in the GUI widgets receive the values and update the interface. This bidirectional communication flow allows the user interface to reflect

real-time changes based on interactions with the underlying Logic module.

The installation process for the qudi Python framework is detailed in the Appendix B

4.6 quEDU-NV Python Graphical User Interface Design

The quEDU platform serves as a sophisticated tool for conducting a variety of quantum physics experiments. It contains single photon detectors and time tagging electronics for data acquisition and analysis as well as digital interfaces to control the experiments. The quEDU and its experiment boards combine the latest achievements in quantum optics technology into easy-to-use system for academic, research and applied purposes with high precision.

The NV experiment board allows users to experience the various properties of nitrogen vacancy centers in diamond, as previously detailed in this report. At the heart of the NV is a green laser focused on the nitrogen-doped diamond through a microscopic objective. The diamond begins to fluoresce in the red wavelength range. This light is collected by the objective and, after some filtering, is coupled into a fiber. The fiber is connected to the quEDU and the fluorescence can be analyzed.

The diamond sample itself is in close proximity to a microwave antenna and is surrounded coils producing a user defined homogeneous magnetic field. In addition, the quEDUs included pattern generator is precisely controlling pulse sequences for laser, microwave, and readout, enabling a multitude of spin-control experiments.

This section delineates the final Python graphical user interface (GUI) developed for the quEDU-NV devices within the qudi framework. The GUI has a toolbar and menu sections to navigate through different experiments.

4.6.1 Home Screen

The homepage provides essential device information for the connected quEDU system. Connection to the device is established through Ethernet, and the application retains IP addresses for streamlined future connections. The Device Info dockwidget shows key details, such as the serial number of the connected quEDU device and current temperature readings. It also indicates if additional devices are connected to the quEDU. In this setup, the application is designed for quEDU-NV, with the quNV add-on linked to the main system. Additionally, the quEDU-NV Temperatures dockwidget displays temperature data for various electronic and mechanical

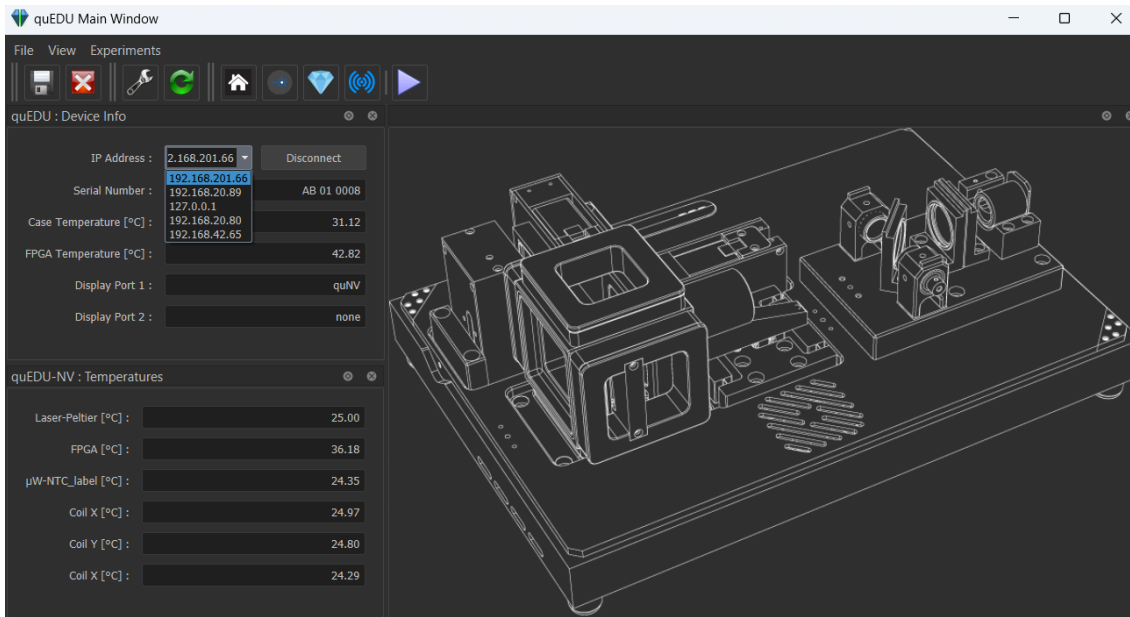


Figure 4.8: Home Screen

components within the quNV setup. Additionally, the dockwidgets offer a flexible interface: they can be repositioned across the screen to suit user preferences. Users can also remove or view specific dockwidgets as needed, allowing them to focus on the most relevant information.

4.6.2 Laser properties

This section presents the Laser controls for the quNV device. The intensity plot serves as a visual representation of the laser intensity.

The Green Laser dockwidget facilitates the adjustment of the laser's operational mode, allowing it to be configured for either continuous mode or Sequence/Pulse mode. The accompanying laser intensity plot illustrates the transition of the green laser—from Sequence/Pulse mode to continuous mode. The red plot represents the fluorescence captured by the avalanche photodiodes (APDs) when the green laser interacts with a diamond situated at the heart of quNV. This interaction results in the emission of red fluorescence from the diamond upon exposure to the green laser.

The APDs & Measurement section allows us to manage the APDs effectively. Here, we can enable or disable specific APDs based on whether we want to view their intensity on the plot. Additionally, adjusting the gain of an APD will impact its intensity.

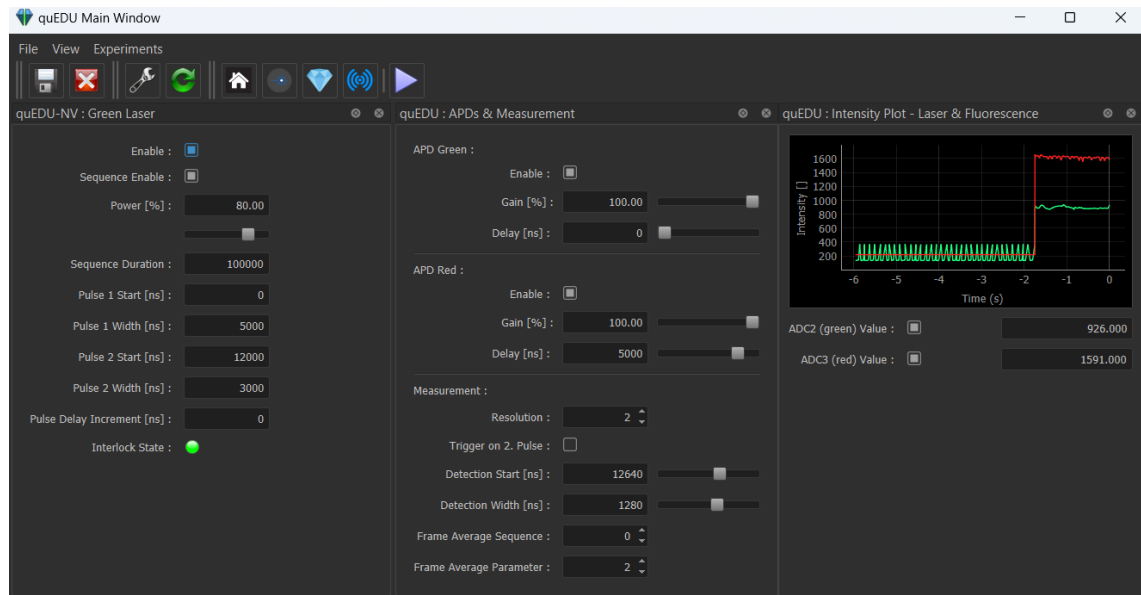


Figure 4.9: Laser Properties

4.6.3 Confocal Alignment

When the green laser interacts with the nitrogen-doped diamond, it produces a high-intensity red fluorescence. In the quNV setup, motors control the movement of the microscopic objective, which emits the green laser. This objective scans along the x, y, and z axes to locate three distinct diamond particles. This precise positioning is essential for further analysis of the NV centers' properties within the diamond.

There are predefined limits established within the server/FPGA that dictate the movement range of the motors. These limits are illustrated by the red rectangle on the XY-image plot. The blue rectangle represents the "Region of Interest," indicating the area where the motors conduct their scanning. Notably, when there is a high intensity (represented as red, grey, or blue based on the selected color), the corresponding pixel will appear vividly bright on the XY-image plot, thereby indicating the presence of a diamond.

The Motion Control and Diamond Scan dock widgets facilitate the maneuvering of the motors. The Motion Control dock widget presents the current positions of the motors, while also allowing users to specify target positions. A red cursor in the shape of a target is displayed on the XY-image plot, indicating the desired position of the motors, which can be established within the Motion Control dock widget. Upon clicking the Start button in the Motion Control dock widget, the green circle cursor on the XY-image plot, representing the current position, will transition to the target position.

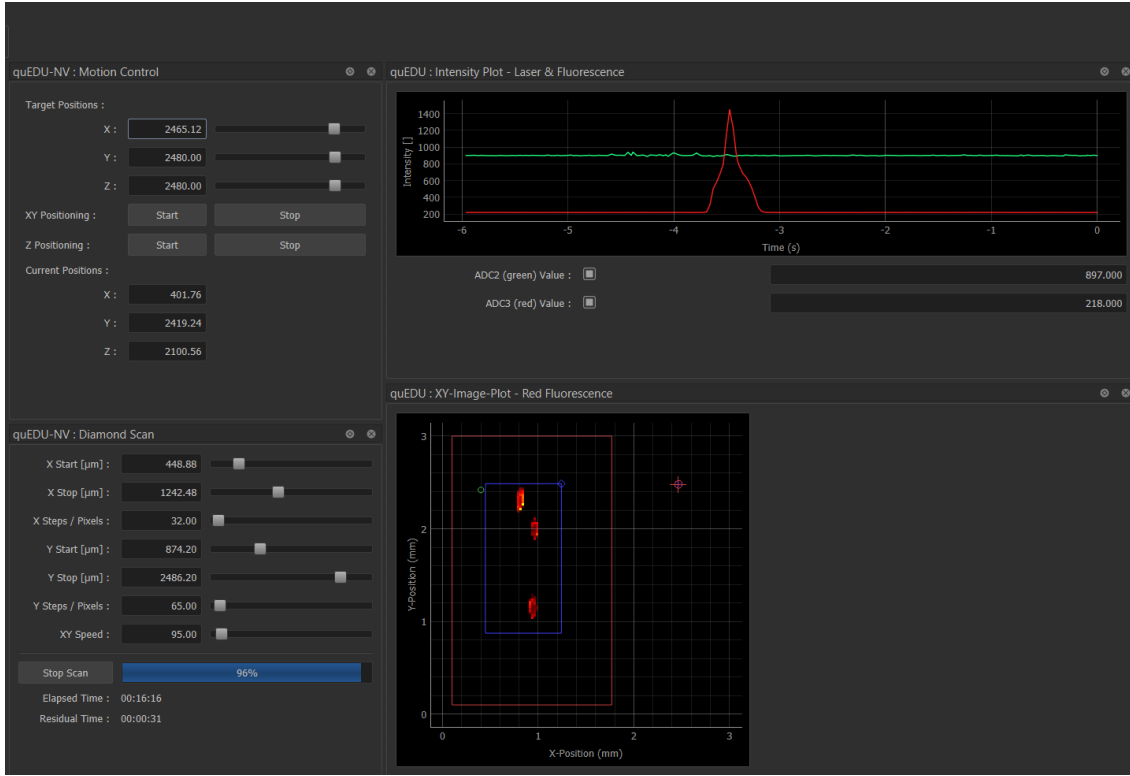


Figure 4.10: Confocal Alignment

The scanning process for locating diamonds can be conducted within the Diamond Scan dock widget. Control over motor movements along the X, Y and Z axes is facilitated, and users have the option to configure the steps or pixels between the initial and final positions. Additionally, this dock widget provides information regarding the total duration of the scan as well as the elapsed time.

The option to designate target positions based on diamond pixels is also available. We can select pixels where a diamond has already been identified from the XY-Image plot, which correspond to regions of higher red fluorescence intensity. This is illustrated in the image below. In the example provided, the red target cursor is positioned on a diamond pixel, and the "start" option has been selected in the Motion Control dock widget. Consequently, the X, Y, and Z coordinates are established for that specific diamond pixel. The elevation in red intensity in the plot further corroborates this observation.

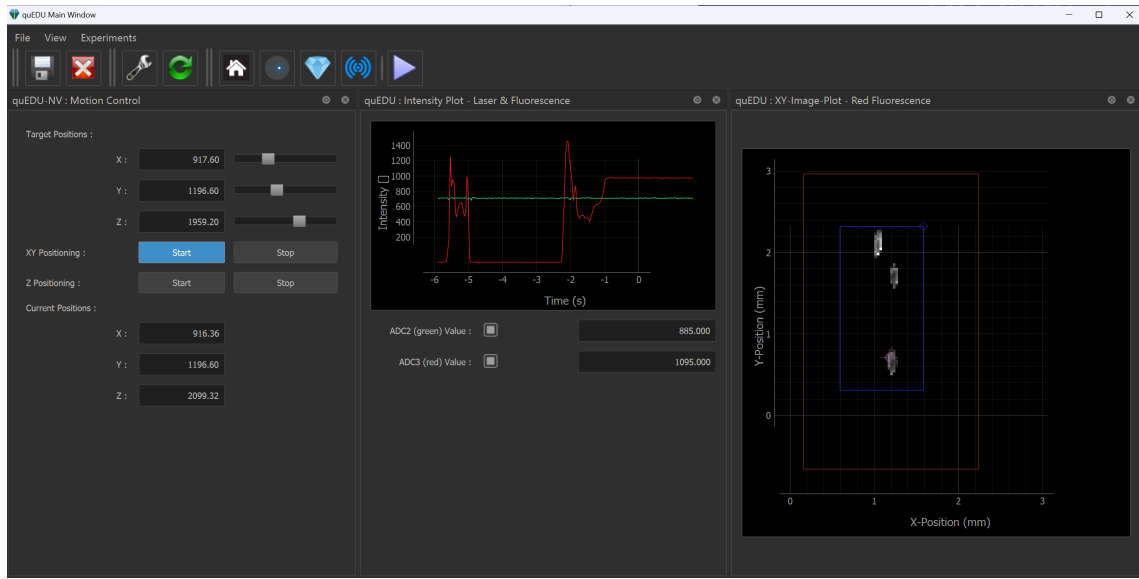


Figure 4.11: Target positions are set based on the Diamond identified pixels

4.6.4 Optically Detected Magnetic Resonance (ODMR)

ODMR allows users to measure magnetic resonance of NV centers by using a combination of optical and microwave fields. The red fluorescence from the quNV changes depending on the NV center's electronic spin state, simultaneously, when a microwave field is applied to the NV centers, and the microwave frequency matches the resonance frequency of the NV center's spin states (typically determined by the local magnetic or electric fields), a drop in red fluorescence intensity is observed.

This fluorescence drop, or "dip," within the ODMR spectrum facilitates the identification of the resonance frequency, directly correlating with the strength of the local magnetic field or the accompanying temperature. The resonance frequencies associated with NV centers are notably responsive to variations in magnetic fields and temperature. Therefore, by conducting a thorough analysis of these fluorescence dips, quNV systems can adeptly measure environmental parameters with exceptional sensitivity.

ODMR is particularly advantageous in the realm of quantum sensing applications, wherein precise measurements of magnetic fields at the nanoscale are essential, particularly in fields such as materials science, biology, and fundamental physics.

The icon located adjacent to the diamond icon in the toolbar represents the ODMR section. Several presets are available concerning ODMR. It is imperative that the laser operates in pulse or sequencer mode rather than continuous mode. Additionally,

the Pulse Enable option must be activated within the Microwave dock widget. Two plots, namely the Oszi-T and Oszi-P plots, allow for the visualization of the pulse mode of the green laser as well as the ODMR spectrum, represented by the dip in red fluorescence.

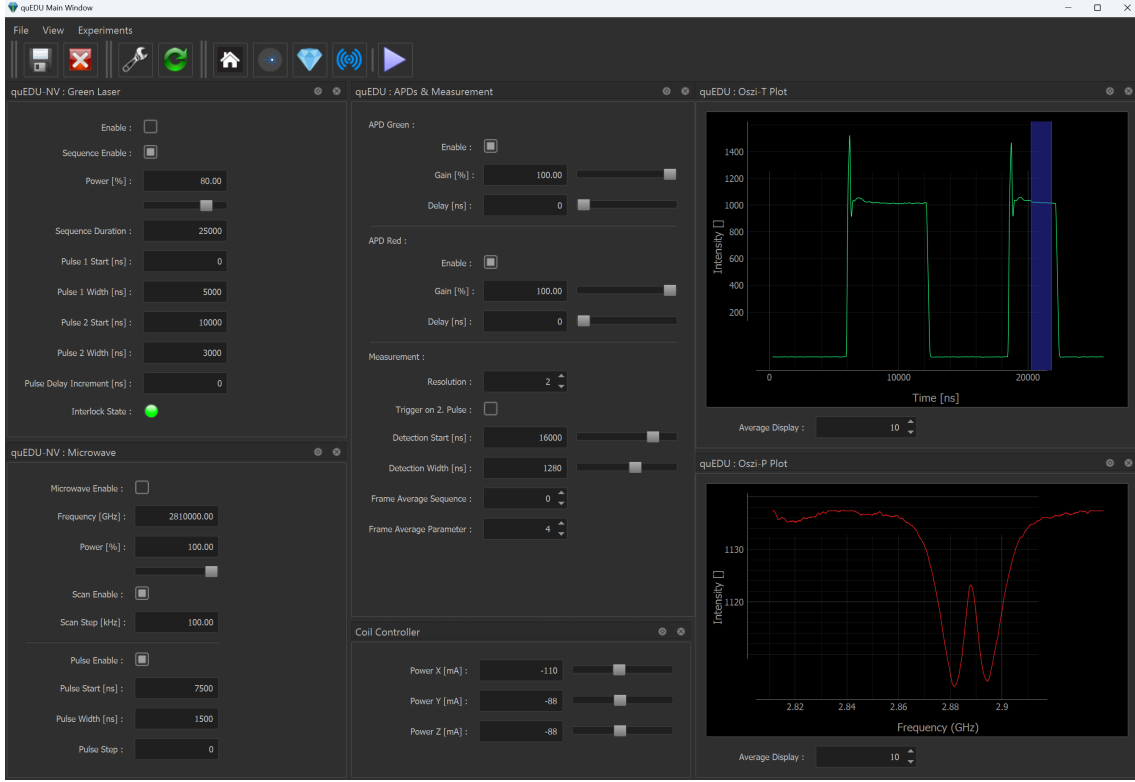


Figure 4.12: ODMR visualization

The blue rectangle on the Oszi-T plot delineates the detection window within which an ODMR dip is observed in the Oszi-P plot. Users have the option to increase the detection width and adjust its position through the APDs and Measurement dock widget. The parameters for average display and resolution can be utilized to reduce noise within the spectrum, thereby yielding a smoother representation of the plot. Furthermore, by varying the microwave power—accomplished through adjustments in the current of the coils via the coil controller dock widget—one may observe multiple dips within the Oszi-P plot. This is illustrated in Fig 4.6

Part II

Appendix

A Experimental Realization of Bell's Inequality with the Quantenkoffer

A.1 Necessary Components

- Quantenkoffer
- 4 Periscopes
- 2 Polarizers

A.2 Experimental description

- **Preperation:** Please plug in the two polarizers as shown in fig below. The source has to be adjusted to obtain the best possible entanglement with the periscopes.
- **Bell experiment:** Set the polarizers to one of the 16 settings listed on the worksheet by touching the respective polarizer and set it to the "Follow" Mode. In the LineView, you can then read off the coincidence count rate for that setting. After you've done that for all 16 settings, you can calculate the S-value.
- **Other States:** For comparison of the results with other input states, try setting the wave plate in the source to another angle. By doing that you can either produce another entangled state (set to M), but you can also produce product states (set to H or V). With these states, you can repeat all measurements. For convenience, there is also the BellView, in which you can measure all 16 settings in a quick and easy way. Depending on the value for the integration time, the measurement is going to be either faster or more precise

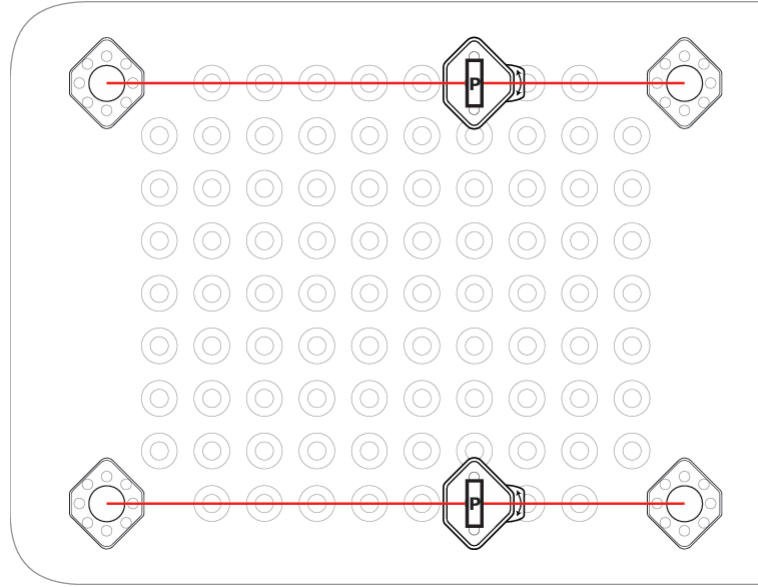


Figure A.1: For the Bell Experiment, put one polarizer in each beam path. You then set the polarizers to different angles and record the coincidence count rate for each setting

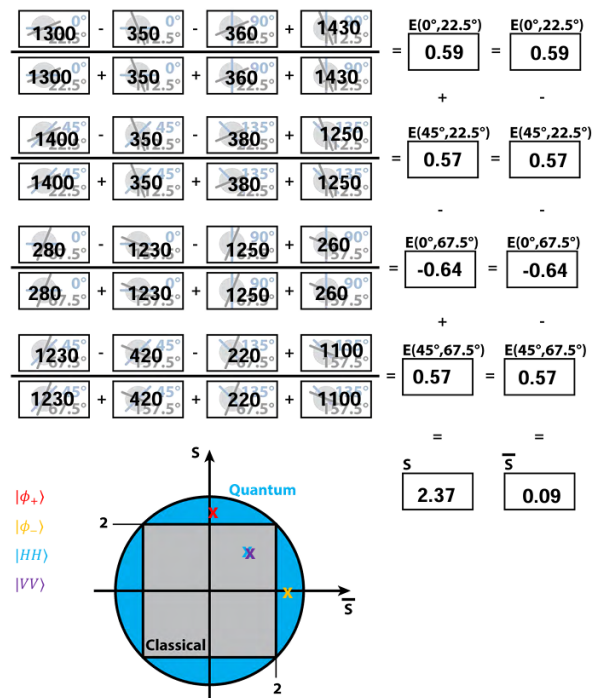


Figure A.2: Worksheet

B quEDU Python GUI Software - Setup

B.1 Based on Python Version 3.10

The quEDU(-NV) Python software is developed using Python Version 3.10, with the objective of ensuring compatibility with the qudi-Framework. However, it is also capable of being executed as a standalone application.

- **Install Python 3.10**

B.2 Update installation tools

- `python.exe -m pip install --upgrade pip setuptools wheel`
- Also update basic packages like numpy, pip, ... etc

B.3 qudi framework

If you like to use the qudi framework, it's best to create an virtual python environment venv, based on python 3.10, in a folder you like for to install the qudi framework in:

- `virtualenv venv --python=python3.10`

Activate this virtual python environment :

- `./venv/Scripts/activate(.bat or .ps1)`

You've got the prompt :

- `(venv)`

Update also the installation tools in the virtual python environment :

- **(venv) python.exe -m pip install --upgrade pip setuptools wheel**

There are several QUDI frameworks, also old and pure ones, in the Web, but the best package seems to come from the IQO institut of Ulm :

- **(venv) pip install qudi_iqo_modules**

Copy the quEDU Python Software Files into the qudi lib folder. Change the path inside of the install batch file and call:

- **install_modules.bat**

Some modules should be installed in qudi folders, like GUI, Logic, Interface and Hardware, but in qutools subfolder.

Copy the quEDU.cfg file to the place, where qudi takes this files (User area) :

- **copy quEDU.cfg ...**

Call qudi framework inside of the virtual environment and load the quEDU.cfg file:

- **(venv) qudi**

B.4 Continue without qudi framework and virtual python environement

Without qudi installation, you need additional python libs (numpy, pyqtgraph, PySide2) for the standalone GUI. They are listed in the requirement file. You can install these libs by taking this requirement file

- **pip install -r requirements.txt**

B.5 Start quEDU Python Software

In qudi the quEDU Python Modules are started inside of the qudi framework. As standalone App, you call the quEDU_Main.py module. This modules do some configuration, which are done in qudi by the qudi framework

- **py quEDU_Main.py**

Bibliography

- [1] ALBERT EINSTEIN, Nathan R. Boris Podolsky P. Boris Podolsky: *Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?* <https://journals.aps.org/pr/pdf/10.1103/PhysRev.47.777>. Version: 1935
- [2] BELL, J.S.: *ON THE EINSTEIN PODOLSKY ROSEN PARADOX**. https://cds.cern.ch/record/111654/files/vol1p195-200_001.pdf. Version: 1964
- [3] FESTIVAL, World S.: *Your Daily Equation: Episode 21*. <https://www.worldsciencefestival.com/videos/your-daily-equation-episode-21/>
- [4] QUTOLS: *quEDU*. <https://qutools.com/quedu/>
- [5] QUTOLS: *quEDU Brochure*. https://www.qutools.com/files/quEDU/qutools_quEDU_Brochure.pdf
- [6] QUTOLS: *quNV*. <https://qutools.com/quedu-nv/>
- [7] QUTOLS: *Quantenkoffer Brochure*. https://www.qutools.com/files/quantenkoffer/qutools_Quantenkoffer_Brochure.pdf. Version: 2021